

SISTEME DISTRIBUITE (DISTRIBUTED SYSTEMS)

Calculatorul paralel reprezintă colecție de elemente de prelucrare care comunică și cooperează pentru rezolvarea rapidă a unor probleme complexe.

Comunicația și cooperarea sunt esențiale într-o arhitectură paralelă.

Axiomele puterii de calcul

- Nu contează cât de rapid este calculatorul cel mai performant de la un moment dat, cineva va dori sa utilizeze unul mai rapid;
- Există întotdeauna probleme importante care cer mai mult decât puterea de calcul disponibilă la un moment dat;
- Cu cât puterea de calcul crește mai mult, cu atât mai multe probleme nerezolvate apar la orizont.

Creșterea vitezei de calcul:

- ***Creșterea performanțelor procesoarelor secvențiale:***
 - Dezavantaje: cost mare al fabricării unui procesor de înaltă performanță, și limitări tehnologice la un moment dat
- ***Parallelism in arhitecturi clasice:***
 - Procesoare pipeline
 - Mai multe UAL
 - Coprocesoare specializate (matematice, grafice, I/O)
- ***Calcul paralel:*** Utilizarea în paralel a mai multor CPU
 - Avantaje: cost - utilizarea mai multor procesoare ieftine fiecare în parte.
 - Dezavantaje: necesitatea de a dezvolta noi medii de lucru, noi algoritmi

Clasificare

Există multe tipuri de sisteme de calcul paralel;
ele se deosebesc prin tipul de interconectare

- între procesoarele componente (cunoscute drept elemente de procesare sau PEs) și
- între procesoare și memorie.

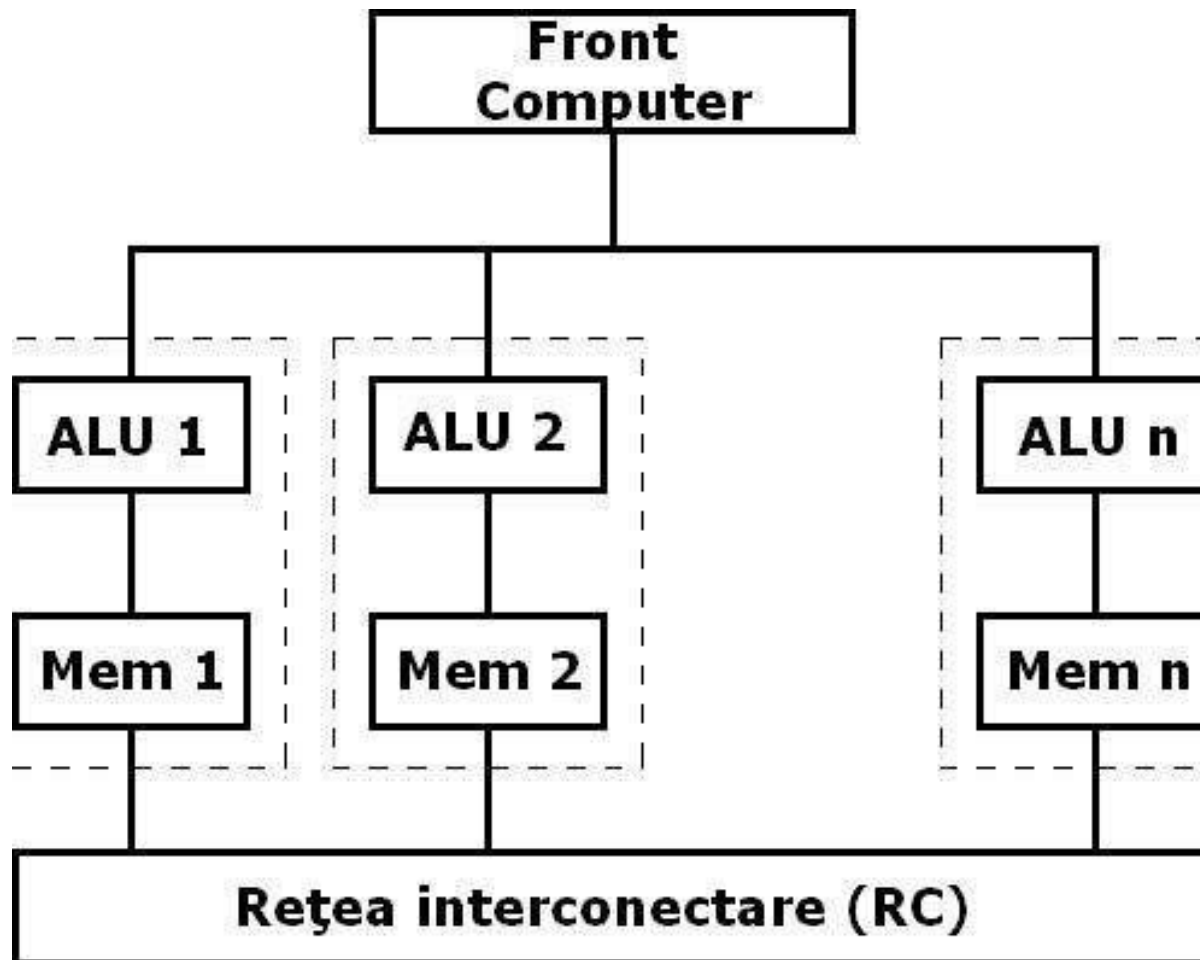
Clasificare (cont.)

Taxonomia lui Flynn (1966) clasifică sistemele de calcul paralel (și serial) astfel: *Single Instruction Multiple Data* sau SIMD (instrucțiune simplă / date multiple): dacă toate procesoarele execută în toate momentele aceeași instrucțiune, dar având ca obiect date diferite

SIMD

- Acestea sunt sisteme cu microprocesoare matriceale, la care operațiile aritmetice se execută în paralel pentru fiecare element al matricei, operația necesitând o singură instrucțiune (se mai numesc și *sisteme de procesare vectorială*).
- Eficiența SIMD-urilor este puternic evidențiată în cazul unor programe care au o cantitate masivă de date paralele reliefate de anumite bucle de programe. Acest tip de arhitectură este folosit în familia de procesoare Intel Pentium MMX (Multi-Media eXtension), care are la bază o arhitectură SIMD. Această familie de procesoare cuprinde un set de 57 de instrucțiuni noi, 4 noi tipuri de date și un set nou de regiștri destinați accelerării execuției aplicațiilor multimedia (grafica 2D și 3D, video, procesare imagine și sunete stereo). Aplicațiile multimedia impun manipularea unor date de dimensiuni mici (8 sau 16 biți) în paralel pentru cele 3 canale de culoare sau cele 2 canale de sunet stereo, cu aceeași instrucțiune, dar date diferite pe aceste canale multimedia.

SIMD



MIMD

Multiple Instruction Multiple Data sau MIMD (instrucțiuni diferite / date multiple): dacă fiecare procesor execută instrucțiuni diferite.

- Sistemele de calcul distribuite folosesc, de obicei, arhitecturi MIMD exploatând fie un spațiu de memorie partajat, fie un spațiu de memorie distribuit.
- Există o extensie uzuală la această taxonomie. Aceasta este:
- Program singular, fluxuri de date multiple (SPMD)- multiple procesoare autonome execută simultan același set de instrucțiuni (dar în puncte independente) pe date diferite. Cunoscut ca și *proces singular, date multiple*.

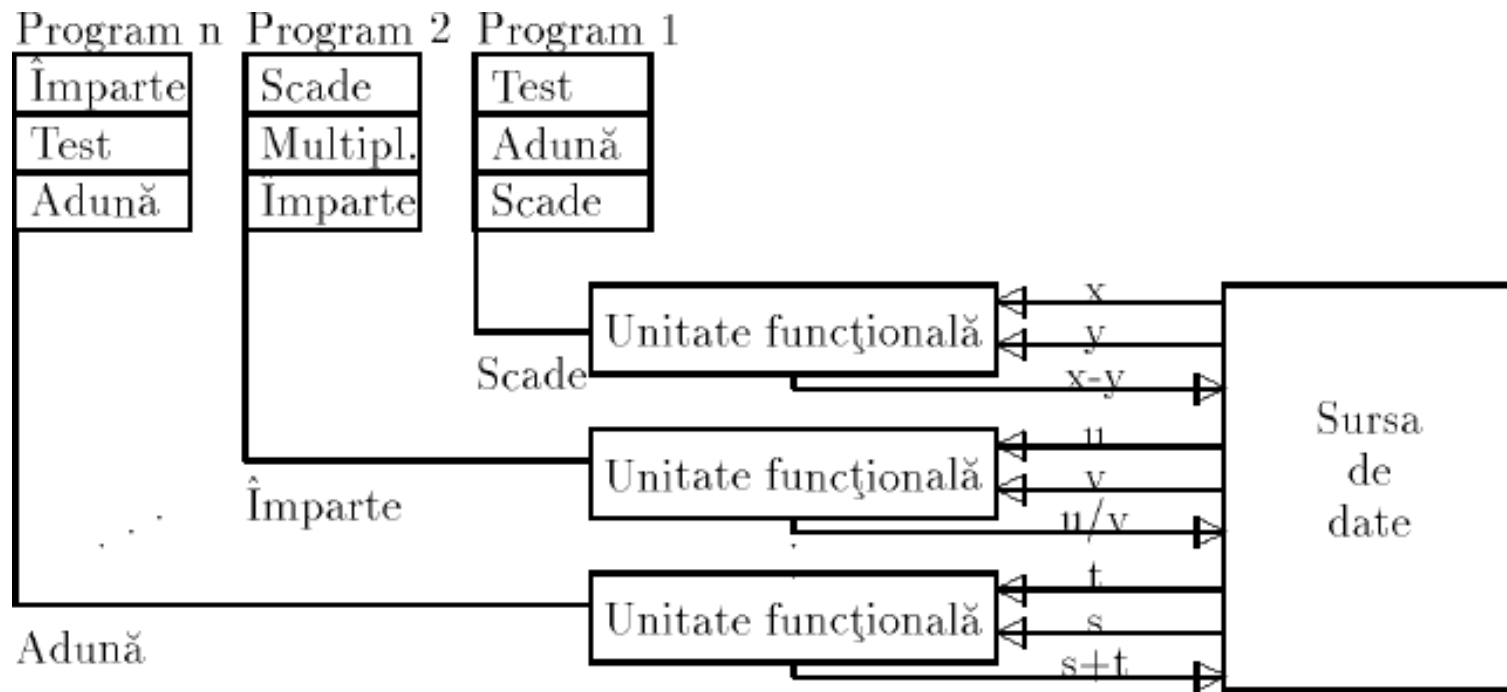
MIMD

- Într-un sistem MIMD, fiecare procesor poate executa diferite operații pe date distincte de cele ale altor procesoare.
- Fiecare procesor funcționează independent de celelalte utilizând propriul contor de program și setul propriu de instrucțiuni.
- Orice comunicare între două procesoare trebuie specificată explicit.

MIMD

- Seturile de instrucțiuni corespunzătoare unor procesoare distincte sunt independente unele de altele, execuția unei instrucțiuni nu influențează execuția alteia și în consecință toate procesoarele pot opera la orice moment.
- Spre deosebire de sistemul SIMD, în MIMD, modul de operare este asincron (fiecare procesor are ceas propriu)

Funcționarea unui sistem MIMD



Clasificare (cont.)

O altă clasificare a sistemelor de calcul paralel este bazată pe arhitectura memoriei:

- Sistemele de calcul paralel cu memorie partajată: dispun de procesoare multiple care accesează toată memoria disponibilă ca un spațiu de adrese global (pentru toate procesoarele). Acestea pot fi împărțite în două mari clase, în funcție de timpul de acces la memorie:
 - Acces uniform al memoriei (UMA), în care timpii de acces la toate părțile memoriei sunt egali
 - Acces neuniform al memoriei (NUMA), în care timpii de acces la memorie nu sunt egali.
- Sistemele de calcul paralel cu memorie distribuită au de asemenea mai multe procesoare, dar fiecare procesor poate accesa doar memoria sa locală; nu există nici un fel de spațiu de adrese global pentru toate procesoarele.

Clasificare (cont.)

- Sistemele de calcul paralel pot fi de asemenea clasificate după numărul de procesoare din componența lor. Sistemele cu mii de asemenea procesoare sunt cunoscute drept *massively parallel*. Urmează sistemele de calcul paralel la scară mare sau la scară mică. Aceasta depinde și de viteza procesorului (de exemplu: un sistem de calcul paralel bazat pe un computer personal va fi considerat "la scară mică").

Clasificare (cont.)

- Sistemele de calcul paralel mai pot fi împărțite în sisteme multiprocesor simetrice și asimetrice, după cum toate procesoarele sunt de același fel sau nu (de exemplu: dacă doar un procesor poate rula codul sistemului de operare și celelalte nu au acest privilegiu, atunci sistemul este asimetric).

Clasificare (cont.)

- Pentru procesarea paralelă au fost realizate o mare varietate de arhitecturi de calculatoare. Spre exemplu o arhitectură de tip "*Ring*" are procesoarele legate între ele într-o structură de tip inel. Alte tipuri de arhitecturi paralele: *Hypercube*, *Fat tree*, *Systolic array* ș.a.

PERFORMANȚE ALE CALCULATOARELOR PARALELE

- **Măsurători de performanță**
- **Factorul câștig de viteză**
- **Principiul mediei armonice ponderate**
- **Legea lui Amdahl**
- **Limitări ale calculului paralel**

Măsurători de performanță

- *Gradul de paralelism DOP* („degree of parallelism”) = numărul de procesoare dintr-un calculator paralel utilizate pe un anumit interval de timp.
- *Profilul paralelismului* = graficul *DOP* în funcție de timp (pentru un anumit program).

Depinde de:

- -structura algoritmului;
- -optimizarea programului;
- -utilizarea resurselor;
- -condițiile de rulare.

Se consideră un sistem paralel cu n procesoare identice.

Paralelism mediu

Δ = capacitatea de calcul pentru un procesor, aproximat cu rata MIPS (million instructions per second):

$$Rata_{MIPS} = \frac{I_C}{T \cdot 10^6} = \frac{f}{CPI \cdot 10^6} = \frac{f \cdot I_C}{C \cdot 10^6}$$

unde

I_C : număr instrucțiuni pentru program („instruction count”);

T : timp de execuție pentru program in secunde;

f : frecvența ceasului in Hz, egala cu $1/\tau$ (τ perioada ceasului în secunde);

CPI : cicluri per instrucțiune (perioade de ceas necesare pentru execuția unei instrucțiuni);

C : număr de cicluri necesare pentru execuția unui program.

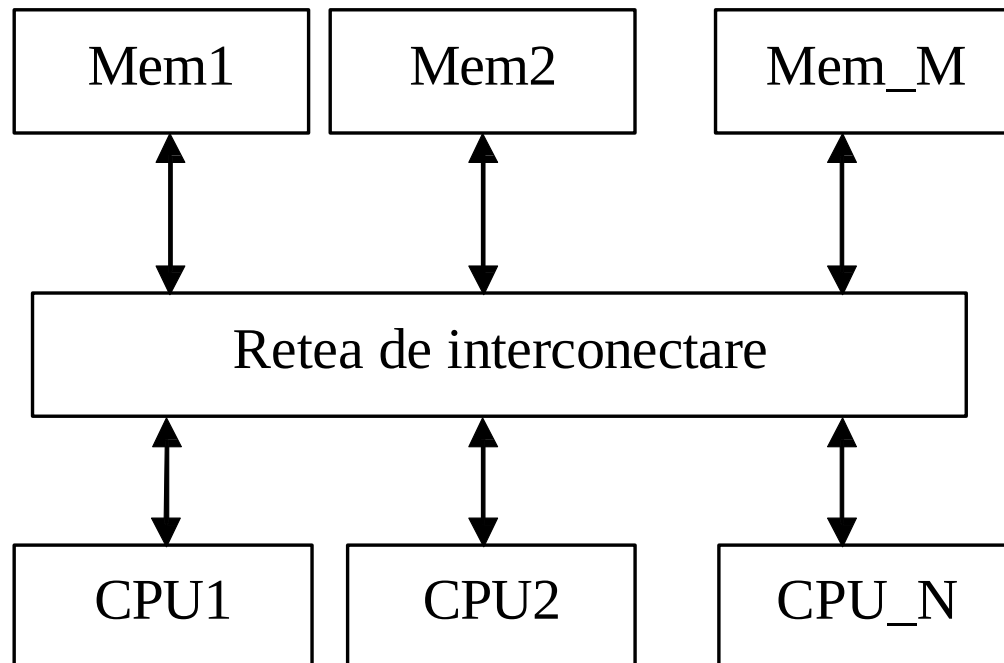
Au fost utilizate relațiile:

$$CPI = \frac{C}{I_C} \quad T = C \cdot \tau = \frac{C}{f} = I_C \cdot CPI \cdot \tau = I_C \cdot \frac{CPI}{f}$$

Aproximarea Δ cu rata MIPS sau rata Mflops nu ține seama de penalizările datorate accesului la memorie, latența comunicației.

Sistem de calcul paralel

“Calculator paralel” = N procesoare + retea de interconectare



Clasificari ale sistemelor paralele

In functie de:

- Mecanismul de control al paralelismului
 - Control centralizat / procesoare independente
- Memorie :
 - Comuna
 - Distribuita
 - Cu spatiu de adrese comun
 - Fara spatiu de adrese comun
- Modul de comunicare intre procesoare
 - Spatiu de adrese comun
 - Comunicare prin mesaje
- Reteaua de interconectare
 - Magistrala
 - Retea de comunicare cu diverse topologii

Clasificarea dupa mecanismul de control

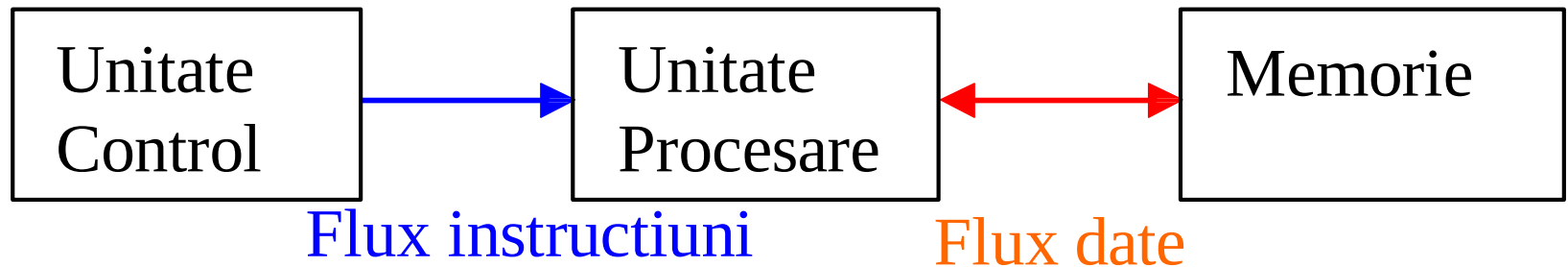
- Controlul sistemului:
 - Centralizat (grupul de procesoare este condus de un controler)
 - Distribuit (fiecare procesor poate lucra independent)
- Clasificarea Flynn:

Dupa natura fluxului de instructiuni (numarul de unitati de control) si a fluxului de date (numarul de procesoare)

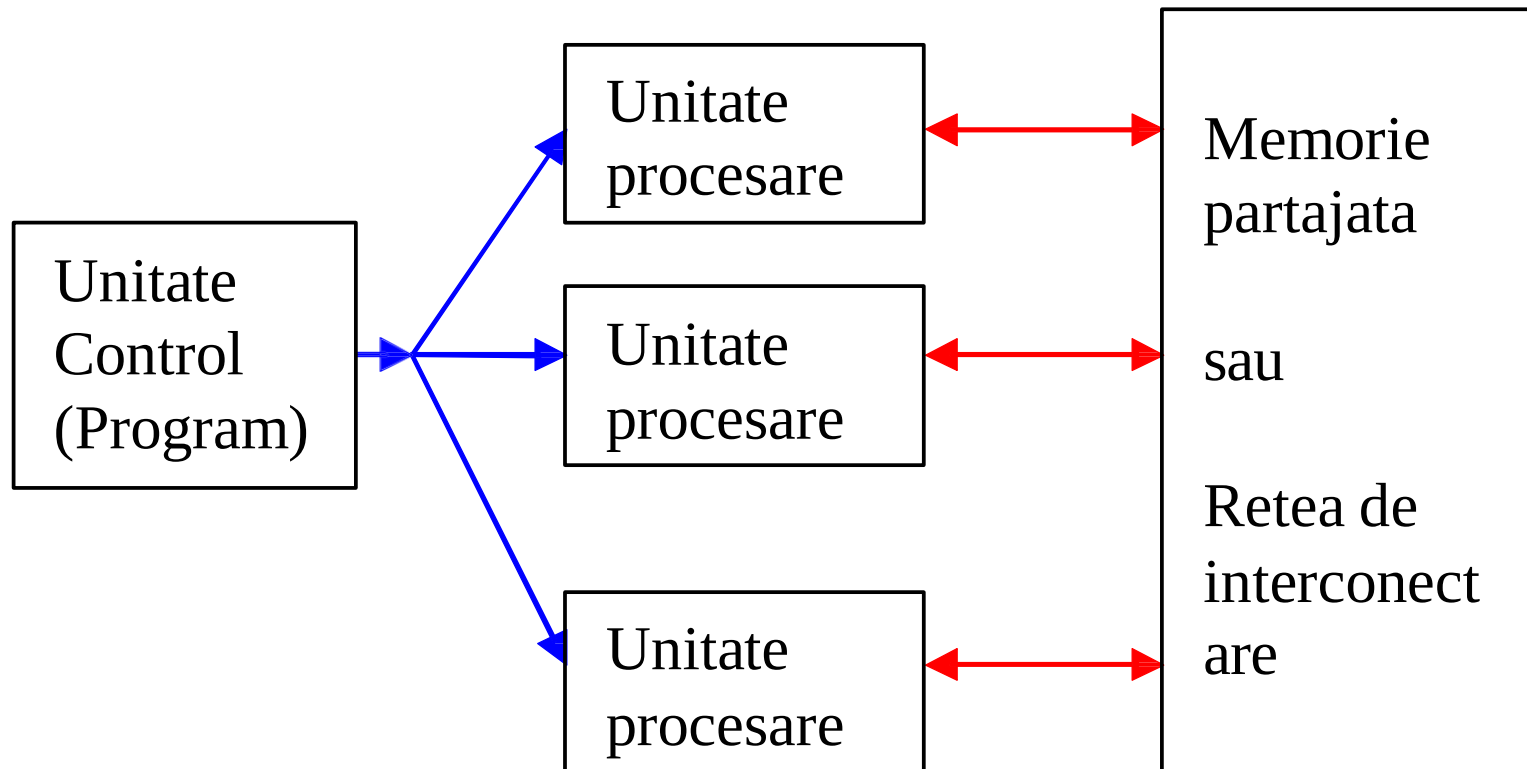
 - SISD: Single Instruction stream, Single Data stream
 - MISD: Multiple Instruction stream, Single Data stream
 - **SIMD**: Single Instruction stream, Multiple Data stream
 - **MIMD**: Multiple Instruction stream, Multiple Data stream

Sisteme SISD

- Masina secventiala clasica, executa o singura instructiune la un moment dat
- Arhitectura Von Neuman



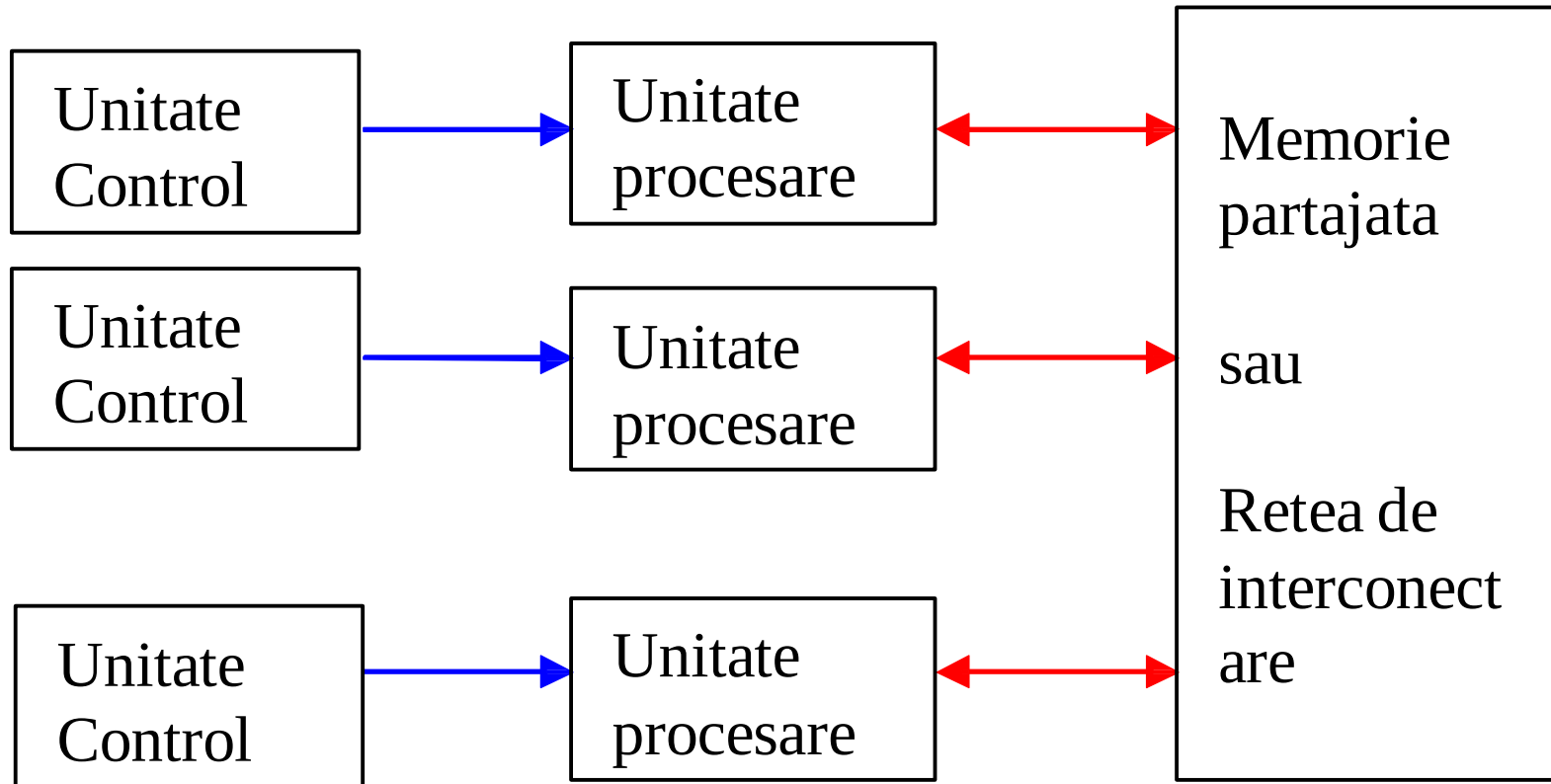
Sisteme SIMD



Sisteme SIMD (cont.)

- 1 unitate de control, N procesoare (identice)
- Unitatea de control transmite **aceeasi instructiune** la toate elementele de procesare
- Fiecare instructiune este **executata in acelasi timp (sincron) pe toate procesoarele**
- Utile pentru paralelismul datelor
- De regula sistemele SIMD au memorie distribuita intre procesoare
- Exemple: procesoare matriceale, masive de procesoare (Illiac IV, Connection Machine CM-200)
 - procesoare vectoriale

Sisteme MIMD

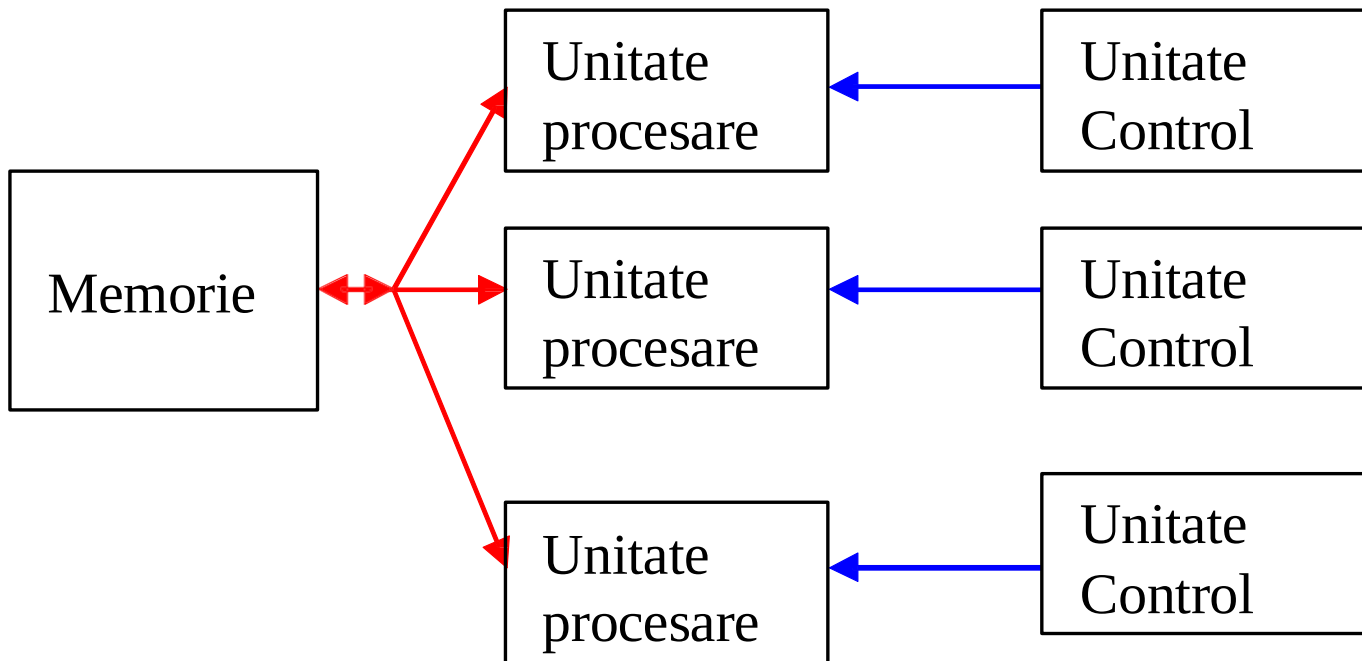


Sisteme MIMD

- Cea mai generala arhitectura
- Fiecare procesor are o unitate de control proprie
- Procesoarele pot opera asincron (executa instructiuni diferite pe date diferite in acelasi timp)
- Comunicarea:
 - Memorie partajata - > multiprocesoare (Cedar, Cray)
 - Retea de interconectare -> multicalculatoare (Paragon, nCUBE)

Sisteme MISD

- N procesoare, N unitati de control (programe), 1 memorie comuna, 1 secventa de date
- Categorie controversata: Nu exista in practica !
- Procesoare pipeline ar putea fi incadrate in aceasta categorie



Retele de interconectare

- Retelele de interconectare furnizeaza mecanismule de transfer de date intre procesoare sau intre procesoare si memorii
- Topologia retelelor de calculatoare:
 - Statica : legaturi directe(point-to-point) intre nodurile procesoare
 - Dinamica : contin legaturi si elemente de comutare (switch)
 - Un element de comutare (switch):
 - set de porturi de intrare si porturi de iesire => gradul sau (numarul total de porturi)
 - Costuri:
 - costul unui switch creste cu patratul gradului sau
 - Costul echipamentelor periferice este direct proportional cu gradul
 - Costul impachetarii este direct proportional cu numarul de pini

Retele cu topologie statica/dinamica

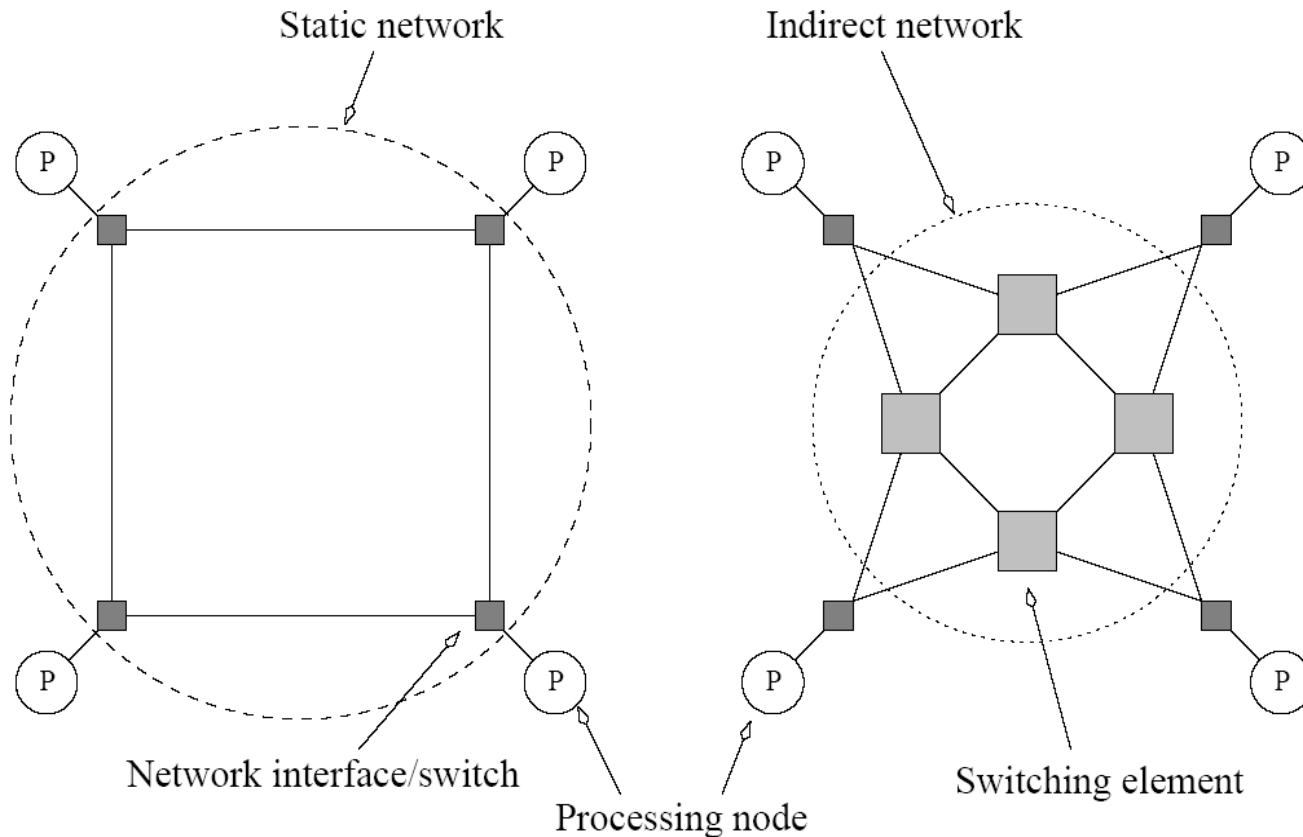


Figure 2.6 Classification of interconnection networks: (a) a static network; and (b) a dynamic network.

Din: [Grama,Gupta,Kumar&Karypis]

Tipuri de topologii

- Retea complet conectata (Completely connected)
- Retea stea
- Masiv linear (Linear array)
- Masiv bidimensional (2D mesh)
- Masiv k-dimensional (k-D mesh)
- Hipercub
- Retea arborescenta (Tree-Based network)
- Magistrala partajata (Bus-based)
- Retea crossbar
- Retea multietape (Multistage network)

Metrice ale retelelor

- Diametrul:
 - Distanța maximă între 2 noduri procesoare
 - Distanța între 2 noduri procesoare se definește ca fiind calea cea mai scurtă (în termenii numărului de legături directe) între acestea
 - De dorit să fie cât mai mic
- Conectivitatea:
 - Numărul minim de drumuri diferite între oricare 2 noduri
 - Numărul minim de arce care trebuie înlăturate pentru a rupe rețeaua în 2 rețele deconectate
 - De dorit să fie cât mai mare
- Latimea biseției (bisection width):
 - Numărul minim de legături care trebuie eliminate pentru a rupe rețeaua în 2 jumătăți **egale** (cvasi-egale dacă rețeaua are un număr impar de noduri)
 - De dorit să fie cât mai mare

Metrice ale retelelor (cont)

- Latimea de banda a unui canal (channel bandwidth):
 - rata maxima de comunicare intre capetele unei legaturi directe
- Bisection bandwidth:
 - $\text{Bisection bandwidth} = \text{bisection width} * \text{channel bandwidth}$
 - De dorit sa fie cat mai mare
- Cost:
 - numarul de legaturi
 - De dorit sa fie cat mai mic

Metrice ale retelelor - cazul retelelor dinamice

- Diametrul:
 - distanta maxima intre doua noduri procesoare
 - Aproximata prin distanta maxima intre oricare doua noduri (procesoare sau switch)
 - Conectivitatea
 - De nod: Numarul minim de noduri (switch-uri) care trebuie eliminate pentru a fragnenta retea
 - De arc: Numarul minim de arce care trebuie eliminate pentru a fragmenta retea
 - Latimea bisectiei:
 - numarul minim de muchii care trebuie eliminate pentru a partitiona retea in doua jumata
- cu acelasi numar de noduri procesoare (nodurile switch nu conteaza !)

Metrice ale retelelor - cazul retelelor dinamice (cont)

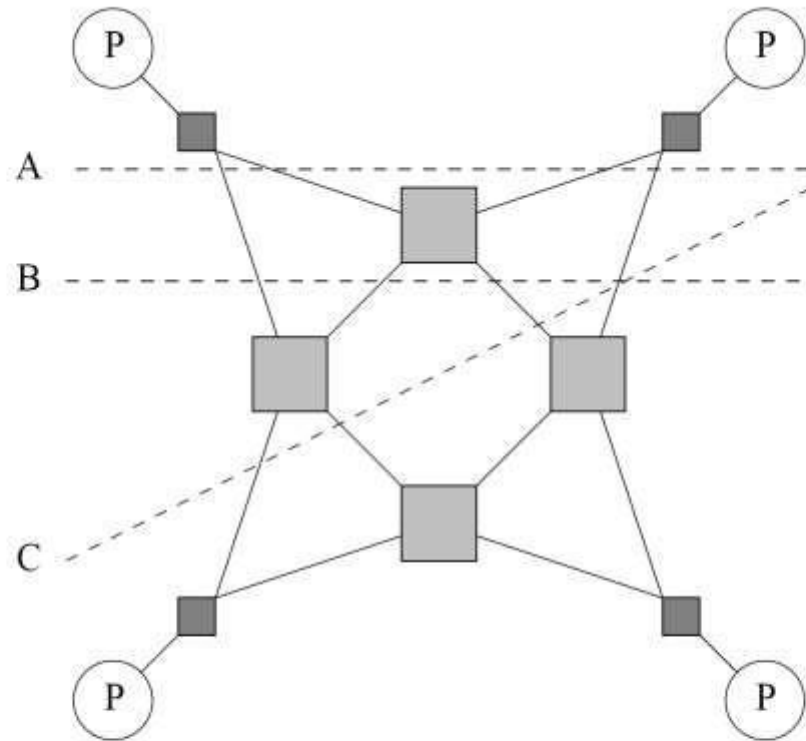


Figure 2.20 Bisection width of a dynamic network is computed by examining various equipartitions of the processing nodes and selecting the minimum number of edges crossing the partition. In this case, each partition yields an edge cut of four. Therefore, the bisection width of this graph is four.

Retea complet conectata si retea stea

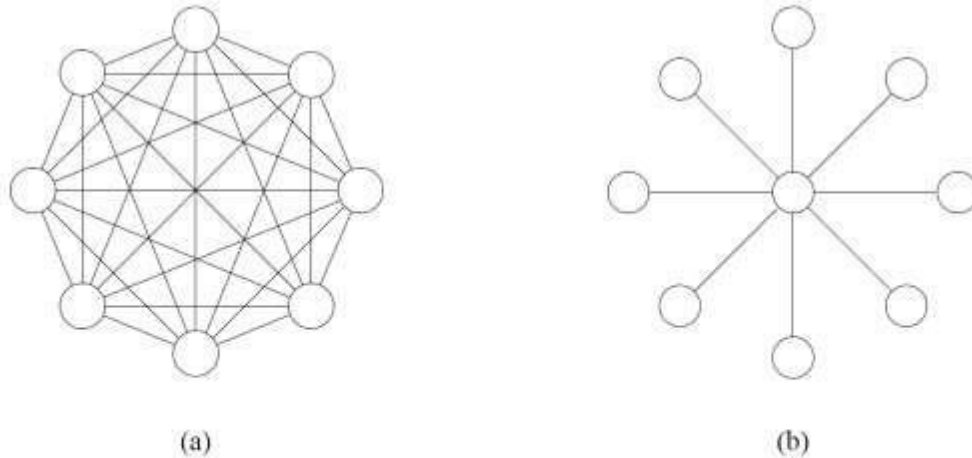


Figure 2.14 (a) A completely-connected network of eight nodes; (b) a Star connected network of nine nodes.

Retea complet conectata

- Fiecare procesor este conectat cu toate celelalte procesoare
- Diametrul = 1
- Conectivitatea = $p-1$
- Latimea bisectiei = $p \cdot p/4$
- Cost : Numarul de legaturi este proportional cu $O(p \cdot p)$
- Complexitatea hardware nu permite realizarea acestui tip de retea pentru valori mari ale lui p
- Asemănătoare cu rețelele crossbar (diferența e că legăturile aici sunt statice)

Retea stea

- Fiecare nod este conectat numai cu un nod comun (“centrul”)
- Diametrul = 2 (Distanța între oricare două noduri este $O(1)$)
- Conectivitatea = 1
- Latimea biseției = 1
- Cost = $p-1$
- Nodul central este punctul slab al rețelei
- Din acest punct de vedere, rețeaua stea este asemănătoare cu magistrala comună

Masive de procesoare: lineare, bidimensionale, k-dimensionale

- Linear: fiecare nod are 2 vecini cu care este conectat direct.
 - Daca sunt legate si nodurile de la extremitati, retea se numeste inel sau tor 1-D
- Masiv 2-D (2D mesh)(grila 2-D): fiecare nod “interior” are 4 vecini (N, S, E, V)
 - Exista si varianta de grila periodica (tor) (wraparound mesh)
- Masiv k-D: prin generalizare, fiecare nod are $2k$ dimensiuni
- **Hipercub**: caz special de masiv k-dimensional: $K = \log p$, unde p este numarul total de noduri

Masive lineare

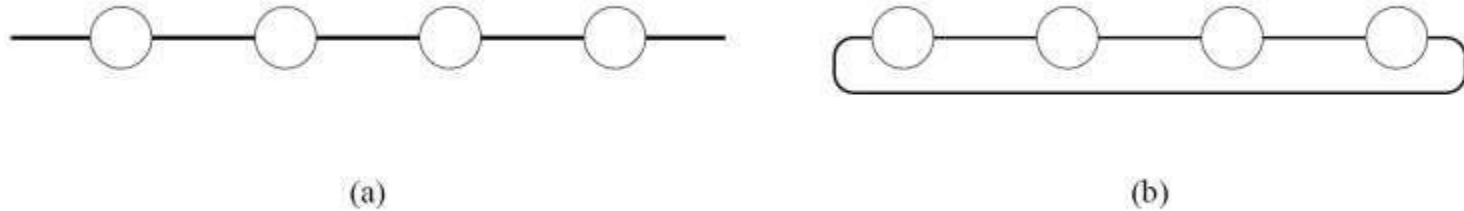


Figure 2.15 Linear arrays: (a) with no wraparound links; (b) with wraparound link.

Masive 2- si 3-dimensionale

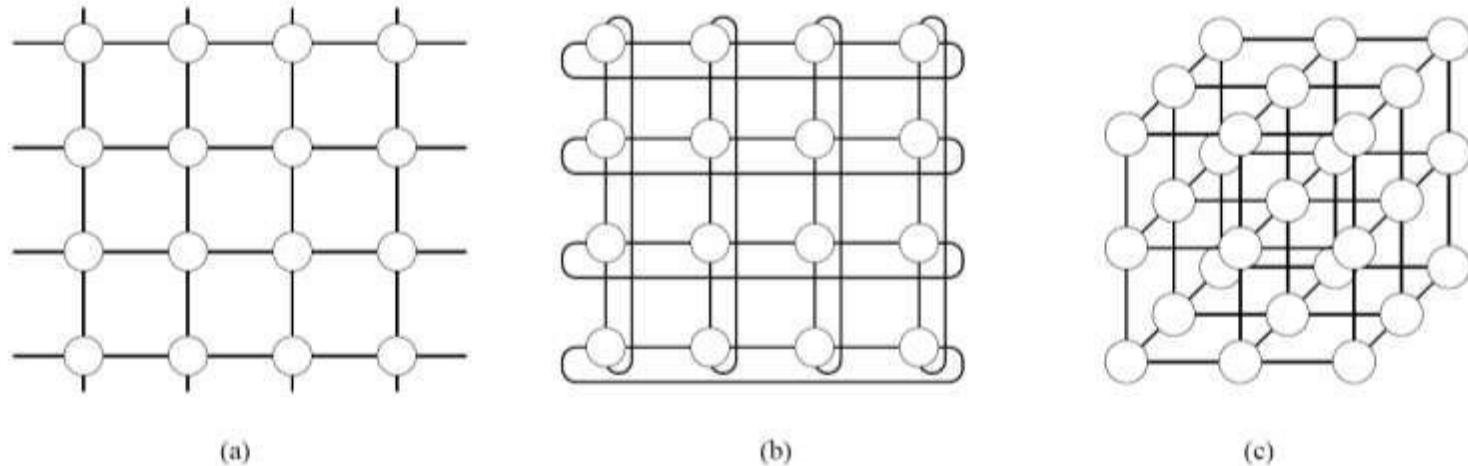


Figure 2.16 Two and three dimensional meshes: (a) 2-D mesh with no wraparound; (b) 2-D mesh with wraparound link (2-D torus); and (c) a 3-D mesh with no wraparound.

Hipercuburi

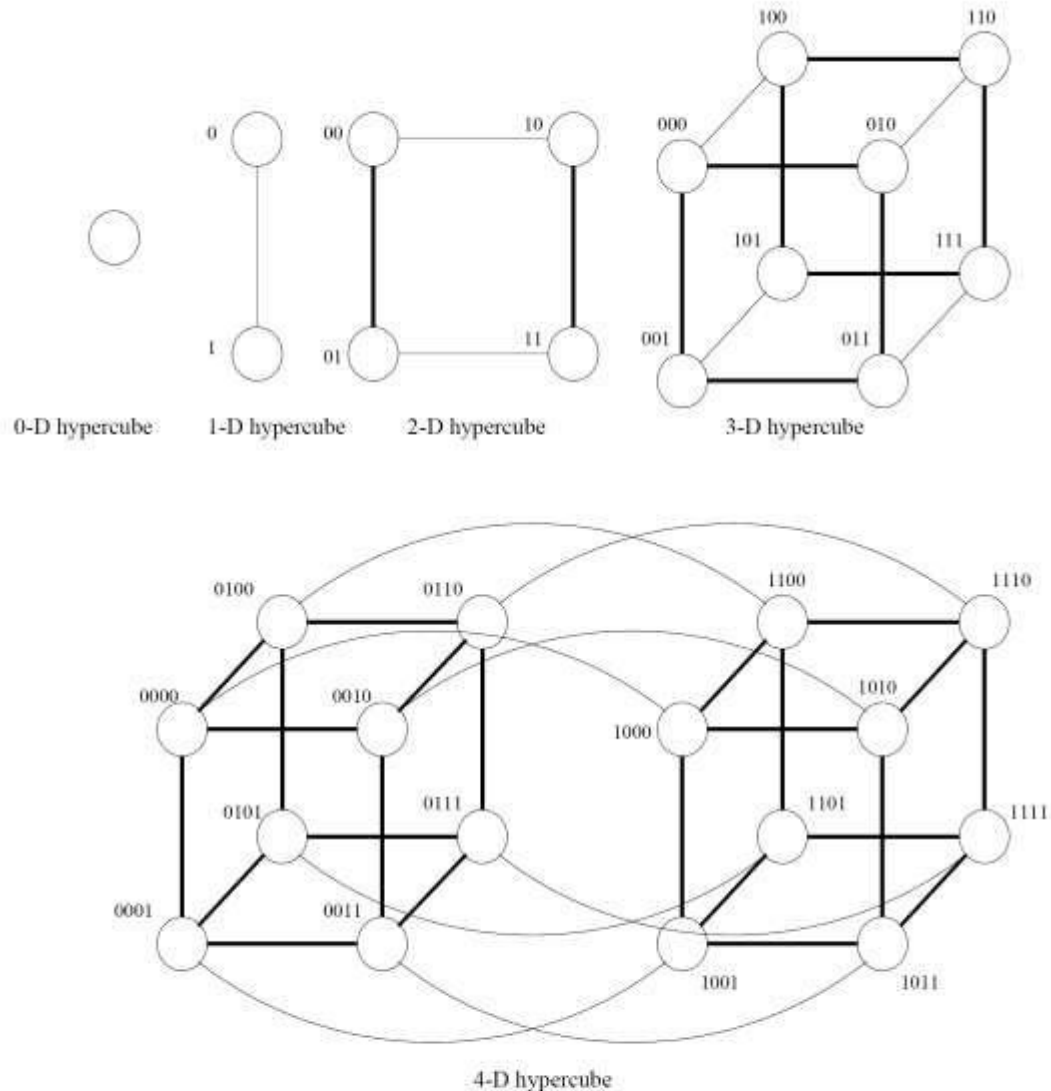


Figure 2.17 Construction of hypercubes from hypercubes of lower dimension.

Din: [Grama, Gupta, Kumar & Karypis]

Hipercuburi (cont)

- Distanța maximă între 2 noduri este $\log p$
- Fiecare nod are $\log p$ vecini
- Distanța între 2 noduri este dată de numărul de poziții binare în care diferă numerele nodurilor (dist Hamming)

Retele arborescente (Tree-Based Networks)

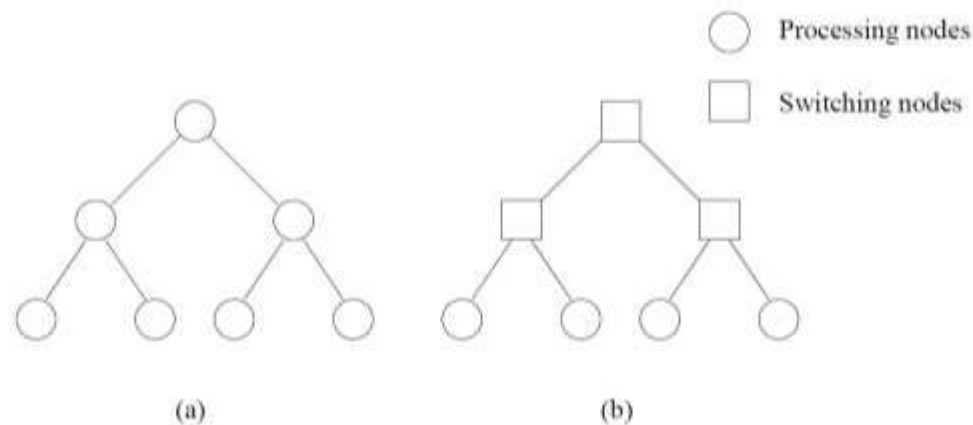


Figure 2.18 Complete binary tree networks: (a) a static tree network; and (b) a dynamic tree network.

Retele arborescente (cont)

- Distanța maximă între două noduri este $2\log p$.
- Legaturile situate mai aproape de rădăcina au un trafic mai mare decât cele aflate mai aproape de bază
- Varianta de arbore gras (fat tree): legaturile mai aproape de rădăcina constau din mai multe canale de comunicare
- Topologiile arborescente sunt planare

Fat Trees

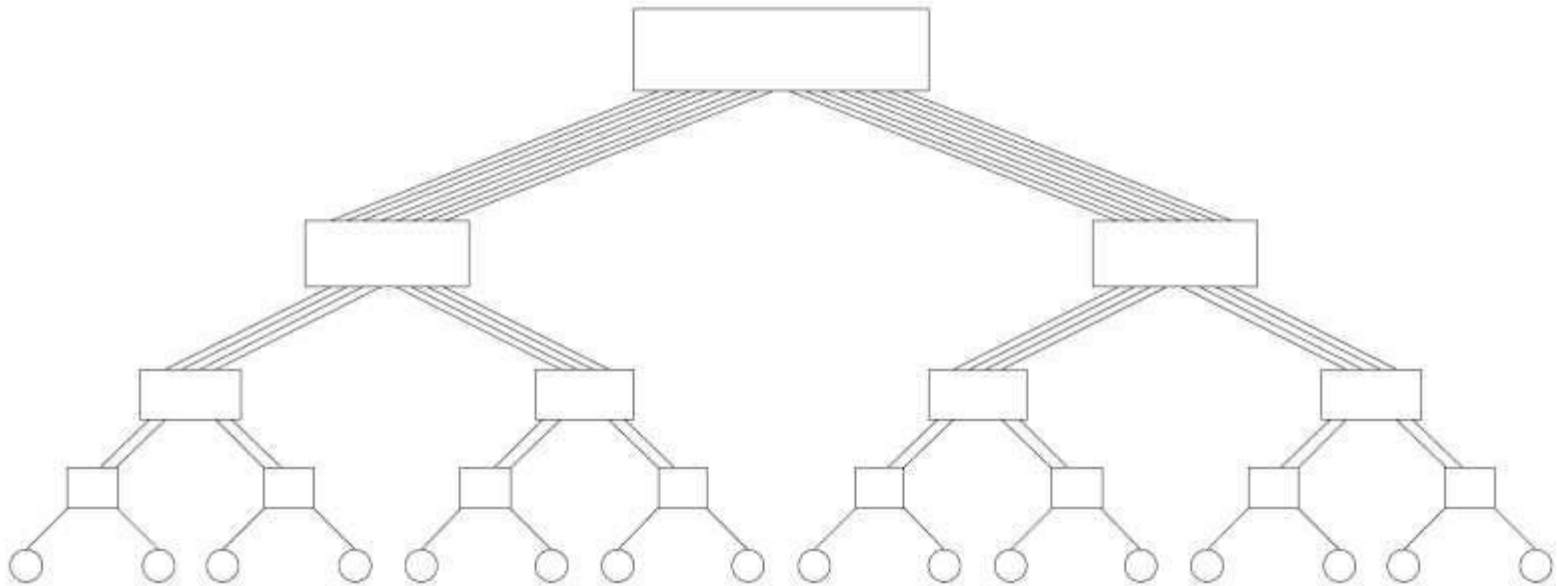


Figure 2.19 A fat tree network of 16 processing nodes.

Comparatie intre retelele cu topologie statica

Table 2.1 A summary of the characteristics of various static network topologies connecting p nodes.

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Completely-connected	1	$p^2/4$	$p - 1$	$p(p - 1)/2$
Star	2	1	1	$p - 1$
Complete binary tree	$2 \log((p + 1)/2)$	1	1	$p - 1$
Linear array	$p - 1$	1	1	$p - 1$
2-D mesh, no wraparound	$2(\sqrt{p} - 1)$	$\sqrt{p}$	2	$2(p - \sqrt{p})$
2-D wraparound mesh	$2\lfloor\sqrt{p}/2\rfloor$	$2\sqrt{p}$	4	$2p$
Hypercube	$\log p$	$p/2$	$\log p$	$(p \log p)/2$
Wraparound k -ary d -cube	$d\lfloor k/2\rfloor$	$2k^{d-1}$	$2d$	dp

Retea crossbar (cont)

- Evaluare:
 - Diametru: $O(1)$
 - Conectivitate: $O(1)$
 - Latimea bisectiei: $O(p)$
 - Cost: $O(p^2)$ $\Rightarrow$ dezavantaj pentru numar mare de procesoare
 - Non-blocking: daca un procesor este conectat la o memorie, aceasta nu impiedica celelalte procesoare sa se conecteze la celelalte blocuri de memorie

Modele si metrice pentru evaluarea performantelor

Overhead in programe paralele

- Utilizand de N ori mai multe resurse in paralel, va rula un algoritm de N ori mai repede ? **NU !**
- Cauze: paralelizarea introduce o regie suplimentara (overhead):
 - **Interactiunea interprocese: comunicare** (transmitere rezultate intermediare sau partiale), sincronizare
 - **Inactivitate temporara (idling):** exista procesoare care sunt inactive pe anumite perioade de timp; Cauze: incarcare inegala, asteptare sincronizare.
 - **Calculare suplimentare:** calcule care nu apar in formularea seriala a algoritmului.

Evaluarea performantelor sistemelor paralele

Modele analitice si metrice

- Metrice de performanta
 - Timpul de Executie T_s si T_p
 - Suprasarcina (Total parallel Overhead) T_o
 - Accelerare (Speedup) S
 - Eficienta E
 - Cost
- Scalabilitatea
 - Legea lui Amdahl
 - Izoeficienta
 - Alte metrice de scalabilitate (scaled speedup)

Timpul de executie

- T_s = Timpul de executie serial: timpul masurat de la inceputul si pana la sfarsitul executiei algoritmului pe un calculator secvential.
- T_P = Timpul de executie paralel: timpul masurat de la inceputul executiei si pana la terminarea ultimului subtask paralel.

Total parallel Overhead

- Suprasarcina (Total parallel overhead) T_o = diferenta dintre timpul total de lucru insumat al tuturor procesoarelor si timpul necesar celui mai rapid algorithm sequential

$$T_o = p \cdot T_p - T_s$$

- Timpul total de lucru insumat al tuturor procesoarelor = $p \cdot T_p$ = timpul de calcul + timpul de comunicare + timpul de inactivitate temporara cumulat pentru toate procesoarele

Accelerarea

- De cate ori mai repede se rezolva o problema in paralel fata de cazul secvential ?
- **Accelerarea (Speedup)** = raportul dintre timpul necesar rezolvarii unei probleme pe un procesor si timpul necesar rezolvarii in paralel pe p procesoare identice

$$S = \frac{T}{T_P}$$

- S este o metrica a performantei algoritmului si nu a calculatorului paralel
- Ts se refera la timpul de executie al ***celui mai bun algorithm secvential*** pentru respectiva problema
 - De exemplu, pentru un algorithm paralel de sortare, se va raporta timpul acestuia la timpul de executia al algoritmului Quicksort secvential, nu Bubblesort secvential !

Valoarea accelerarii

- Teoretic (ideal):
 - accelerare liniara **$S=p$**
- De obicei in practica:
 - **Accelerare subliniara $S < p$** (datorita overhead)
- Posibile anomalii:
 - accelerare supraliniara **$S > p$** .
 - Cauze: imbunatatirea accesului la memorie prin partitionarea datelor de volum mare intre mai multe noduri; comportarea unor algoritmi pe cazuri particulare.

Estimarea accelerarii

$$S(n, p) = \frac{T_S(n)}{T_P(n, p)}$$

- $T_S(n)$ = complexitatea de timp secventiala pentru o problema data
 - Este functie de n =dimensiunea datelor
- $TP(n,p)$ = complexitatea de timp paralela
 - Estimeaza timpul de rulare a algoritmului concurent pe p procesoare; depinde si de modelul arhitecturii paralele
- $T_S(n) \leq TP(n, 1)$

Exemplu: Adunarea a n numere

Varianta 1 - n procesoare

- Initial, fiecare procesor detine in memoria sa locala una din valorile care trebuie insumate
- Fiecare procesor de ordin impar transmite valoarea pe care o detine procesorului din stanga sa care realizeaza suma partiala
- Dupa primul pas -> raman $n/2$ valori (sume partiale) de insumat pe $n/2$ procesoare
- Se repeta procedura, dupa al doilea pas -> raman $n/4$ sume partiale de insumat
- Dupa un numar de $\log(n)$ pasi, suma finala se gaseste la primul procesor

Estimarea accelerarii

- Pentru algoritmul de adunare a n valori utilizand n procesoare
 - Numarul de pasi = $\log(n)$
 - Fiecare pas consta dintr-o operatie de comunicare si o operatie de adunare, de timp constant $t_{comm}+t_{add}$.
 - $TP=O(\log n)$
- Adunarea a n numere - cazul secvential:
 - $TS= O(n)$
- $S=O(n/\log n)$

Eficiența

- Eficiența utilizării calculatorului paralel:

$$E = \frac{S}{p}$$

- $E \leq 1$, este 1 în cazul ideal $S=p$
- Exemplu: adunarea a n numere pe n procesoare:
- $E=S/n=O(1/\log n)$

Exemplu calcul metrici

Serial :

$$T_s = 9 \cdot c \cdot n^2$$

Paralel:

Imaginea este partitionata in p "fasii" de latime n/p

Fiecarefasie trebuie sa comunice coloanele

de valori aflate pe frontierastanga respectivdreapta

$$T_p = 9 \cdot c \cdot \frac{n^2}{p} + 2(t_s + t_w \cdot n)$$

$$S = \frac{T_s}{T_p} = \frac{9 \cdot c \cdot n^2}{9 \cdot c \cdot \frac{n^2}{p} + 2(t_s + t_w \cdot n)}$$

$$E = \frac{S}{p} = \frac{1}{1 + \frac{2p(t_s + t_w \cdot n)}{9 \cdot c \cdot n^2}}$$

Cost

- $\text{Cost} = TP * p$
- Costul reflecta timpul total de lucru insumat al tuturor procesoarelor
- $E = Ts / \text{Cost}$
- Algoritm paralel **optimal in cost (cost-optimal)**:
costul rezolvarii problemei cu ajutorul
algoritmului paralel este egal cu timpul de
executie al celui mai rapid algoritm secvential
 $\text{Cost} = O(Ts(n))$
- Eficienta unui sistem cost-optimal este $O(1)$

Granularitatea

- Granularitate = dimensiunea taskurilor in care e descompusa problema paralelizabila
 - Granularitate fina (fine-grained): numar mare de taskuri mici
 - Granularitate mare (coarse-grained): numar redus de taskuri mari
- Cresterea granularitatii (*scaling down*) (scaderea numarului de procesoare) poate ajuta la imbunatatirea metricii de cost

Exemplu: adunarea a n numere pe p procesoare, varianta 1

- Cresterea granularitatii: $p < n$
- Exista algoritmul prezentat anterior, de adunare a n numere pe n procesoare, in $\log n$ pasi
- Cele n procese (procesoare virtuale) se mapeaza pe cele p procesoare fizice existente: procesul i se mapeaza pe procesorul $[i \bmod p]$, datele se distribuie analog
- Fiecare din cele p procesoare fizice ruleaza n/p procese

Granularitate $\leftrightarrow$ cost optimalitate

- Un algoritm ruleaza pe p_1 procesoare, in timpul paralel T_{p1} , cu $\text{cost1} = p_1 * T_{p1}$
- Se reduce numarul de procesoare la $p_2 < p_1$; p_2 procesoare fizice simuleaza comportarea celor p_1 procesoare logice
- Fiecare procesor fizic: creste volumul de munca cu un factor p_1/p_2
- Comunicatiile intre procesoare: posibil scad; sau cresc dar nu cu mai mult de un factor p_1/p_2
- T_{p2} creste cel mult cu un factor p_1/p_2
- $\text{Cost2} = p_2 * T_{p2} = p_1 * T_{p1} = \text{Cost1}$
- ***Reducerea granularitatii prin simularea proceselor pe mai putine procesoare fizice pastreaza proprietatea de cost-optimalitate, daca sistemul este cost optimal***
- ***Daca sistemul nu este cost-optimal, acest tip de crestere a granularitatii nu ajuta***

Importanta cost-optimalitatii

Exemplu: un algoritm de sortare care utilizeaza n procesoare si ruleaza in timp $(\log n)^2$

- $T_s = n \log n$,
- $S = n / \log n$
- $E = 1 / \log n$
- $Cost = p T_p = n (\log n)^2$.
- Algoritmul nu este cost-optimal, cu un factor $\log n$.
- *Daca se utilizeaza $p < n$ procesoare:*
 - $T_p = n (\log n)^2 / p$.
 - $S = p / \log n$.

Daca se pastreaza constant nr de procesoare p , si se creste dimensiunea problemei n , accelerarea S scade !!! (intuitiv ar fi de asteptat ca S sa creasca)

Metrice ale performantei - rezumat

Metrica	Notatie/formula	Semnificatie
Timp executie secventiala	T_s	Executia celui mai performant algoritm secvential
Timp executie paralela	T_p	
Accelerare	$S = T_s / T_p$	In general $S < p$
Eficienta	$E = S / p$	In general $E < 1$
Cost	$\text{Cost} = p * T_p$	Cost-optimal daca $p * T_p = T_s$
Suprasarcina	$T_o = p T_p - T_s$	

Scalabilitate

- Performanta unui sistem paralel depinde de dimensiunea problemei si de numarul de procesoare
- Ce se intampla daca:
 - Creste dimensiunea problemei
 - Creste numarul de procesoare
 - ***Cum si cu cat se modifica performantele ?***

Cum se pot face extrapolari ale performantei pornind de la dimensiuni reduse ale problemei si dimensiuni reduse ale sistemului (nr procesoare) pentru dimensiuni mai mari ale problemei si nr mai mare de procesoare ?

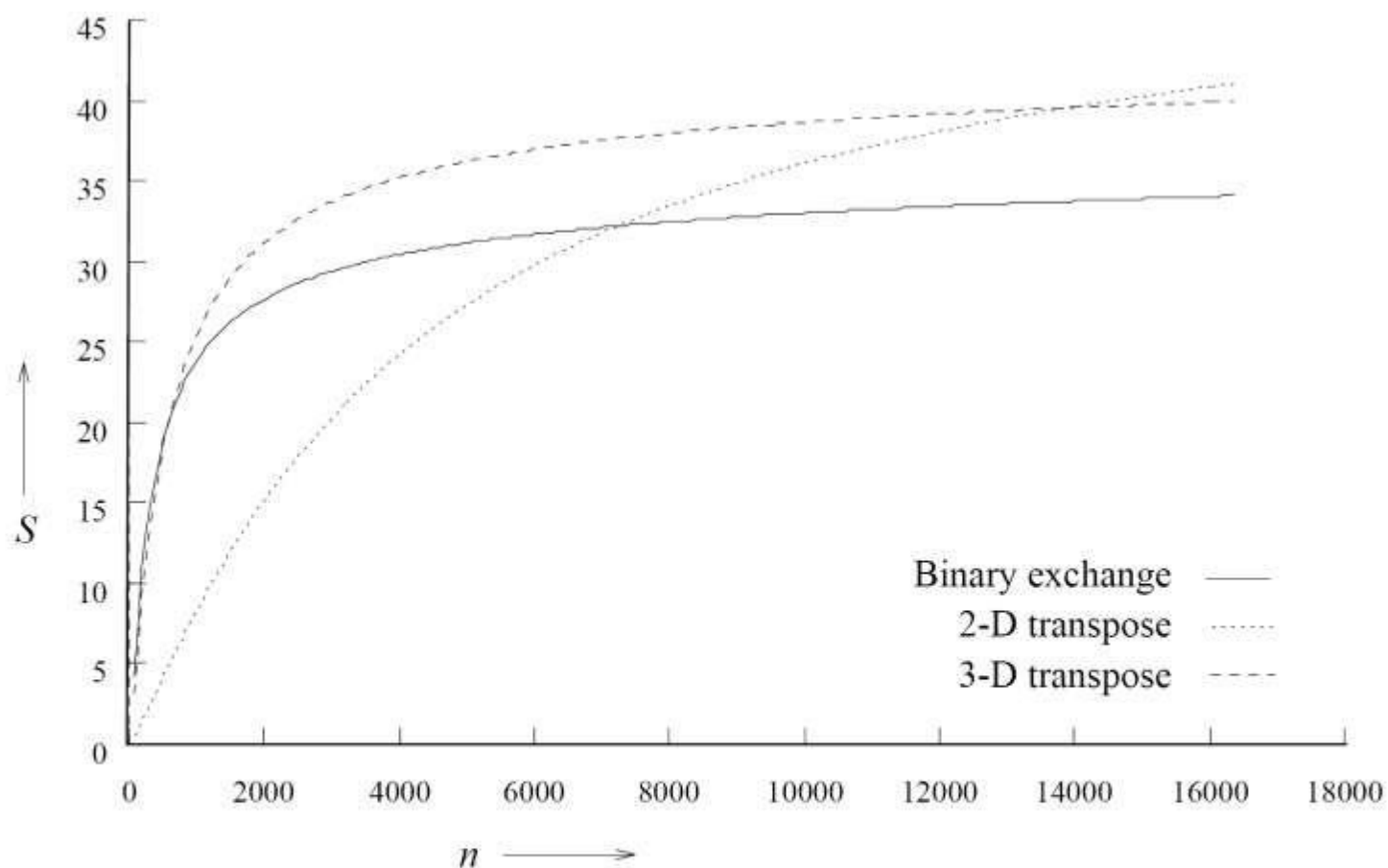


Figure 5.7 A comparison of the speedups obtained by the binary-exchange, 2-D transpose and 3-D transpose algorithms on 64 processing elements with $t_c = 2$, $t_w = 4$, $t_s = 25$, and $t_h = 2$ (see Chapter 13 for details).

Legea lui Amdahl - valoarea limita a accelerarii

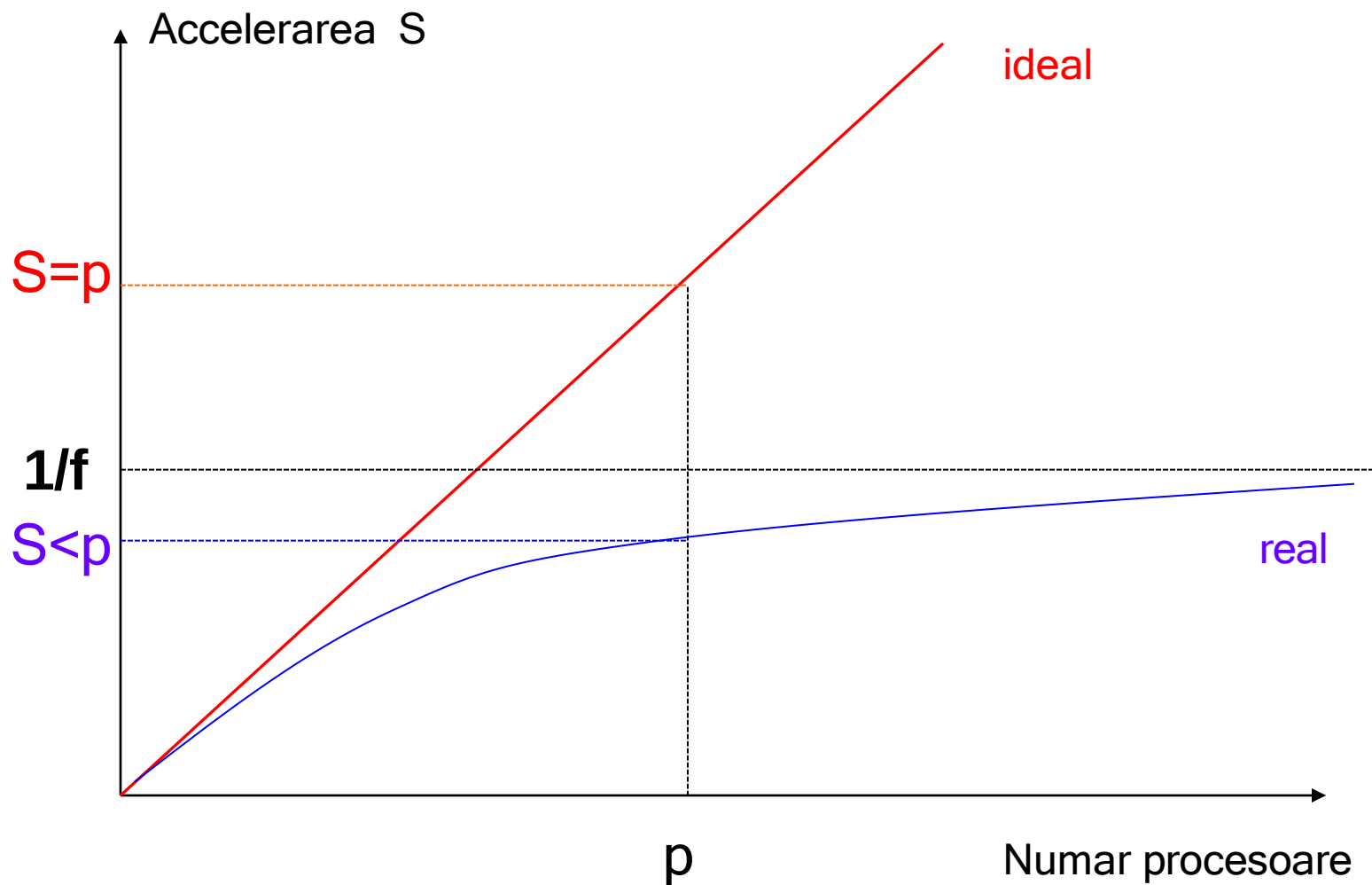
- **Coeficient de secventialitate f :** procentul din calculele unui algoritm care sunt inerent secventiale

$$T_P = f \cdot T_S + \frac{(1-f) \cdot T_S}{p}$$

$$S = \frac{T_S}{T_P} = \frac{1}{f + \frac{1-f}{p}}$$

$$\lim_{p \rightarrow \infty} S = \frac{1}{f}$$

Legea lui Amdahl - Accelerarea functie de numarul de procesoare



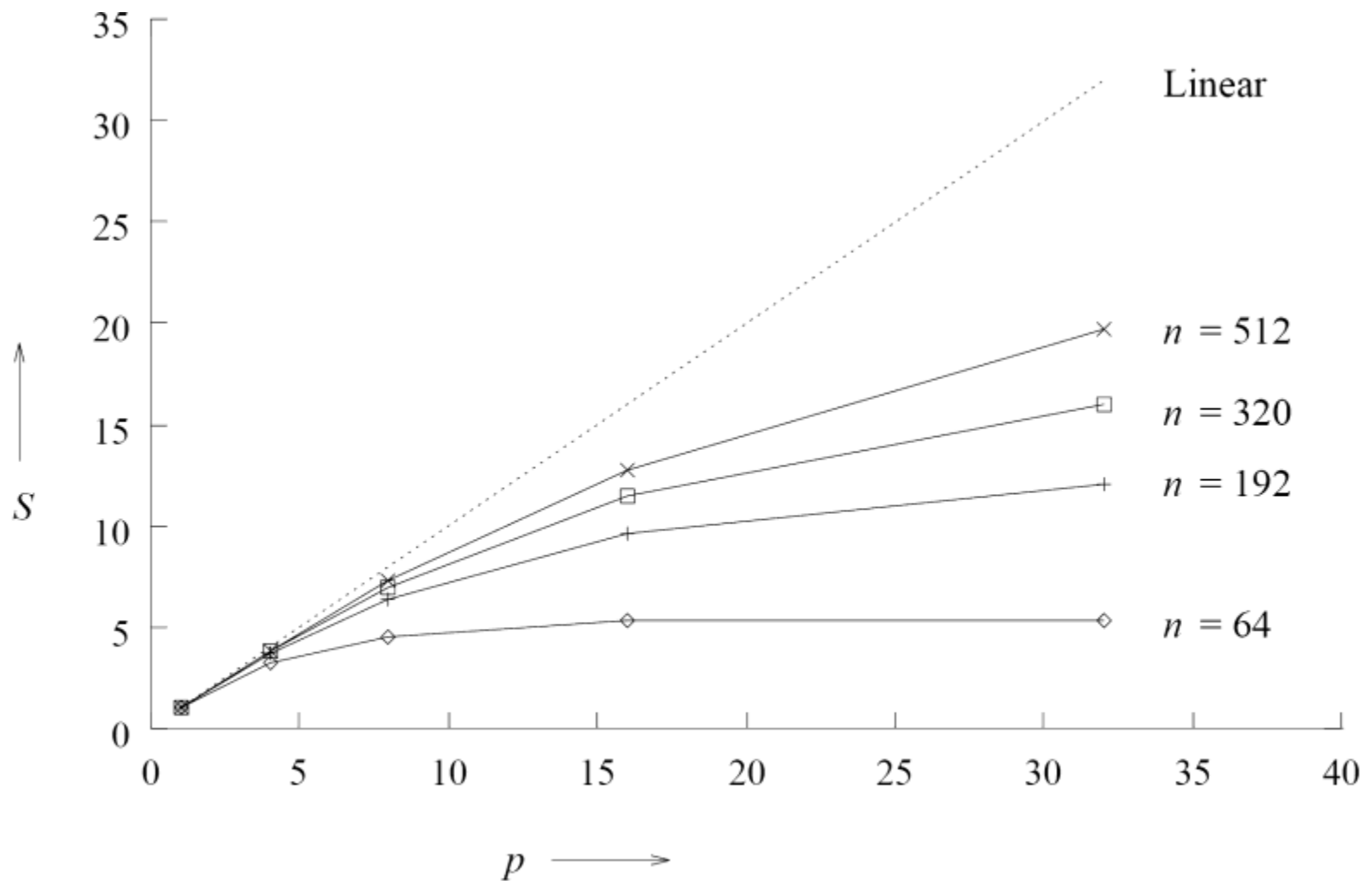


Figure 5.8 Speedup versus the number of processing elements for adding a list of numbers.

Din: [Grama,Gupta,Kumar&Karypis]

Relatia: Eficienta - Numar
procesoare - Dimensiunea problemei

$$E = \frac{S}{p} = \frac{T_S}{pT_P}$$

$$E = \frac{1}{1 + \frac{T_o}{T_S}}$$

$$T_o = p \cdot T_p - T_s$$

$$T_o = T_o(p, T_s)$$

Relatia: Eficienta - Numar procesoare - Dimensiunea problemei (cont)

- Pentru o dimensiune fixa a problemei, Eficienta E scade odata cu cresterea numarului de procesoare p
 - Dimensiunea p_b e fixa $\Rightarrow T_s$ constant
 - Nr procesoare p creste $\Rightarrow T_o$ creste cel putin liniar cu $p \Rightarrow E$ scade
- Pentru un numar fix de procesoare, eficienta E ***poate*** sa creasca daca creste dimensiunea problemei
 - ***Daca T_o are o crestere subliniara relativ la T_s :*** raportul T_o/T_s scade $\Rightarrow E$ creste

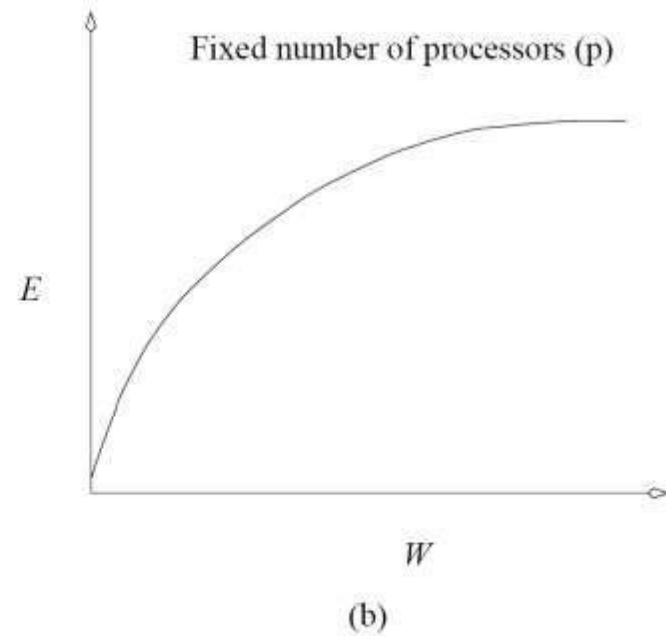
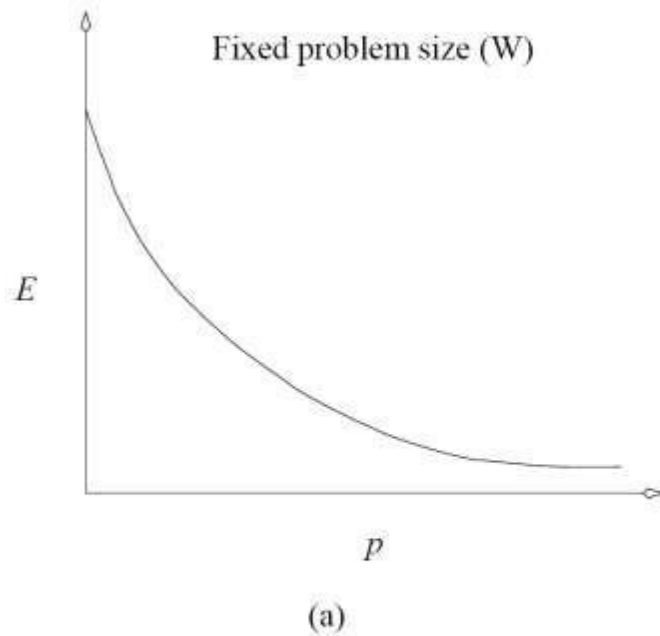


Figure 5.9 Variation of efficiency: (a) as the number of processing elements is increased for a given problem size; and (b) as the problem size is increased for a given number of processing elements. The phenomenon illustrated in graph (b) is not common to all parallel systems.

Definitia scalabilitatii

- **Scalabilitate:** pastrarea eficientei E constante prin cresterea simultana a dimensiunii problemei si numarului de procesoare
- **Un sistem in care este posibil acest lucru se numeste scalabil**
- **Problema:**
 - p creste la p'
 - Cat trebuie sa creasca n asfel incat $E(n,p)=E(n',p')$

Table 5.1 Efficiency as a function of n and p for adding n numbers on p processing elements.

n	$p = 1$	$p = 4$	$p = 8$	$p = 16$	$p = 32$
64	1.0	0.80	0.57	0.33	0.17
192	1.0	0.92	0.80	0.60	0.38
320	1.0	0.95	0.87	0.71	0.50
512	1.0	0.97	0.91	0.80	0.62

Izoeficienta

- **Functia de izoeficienta: *specifica rata de crestere a dimensiunii problemei necesara pentru a mentine constanta eficienta, odata cu cresterea numarului de procesoare***
- Sisteme nescalabile: nu se poate determina functia de izoeficienta (nu exista): eficienta nu poate fi mentinuta constanta la cresterea numarului de procesoare
- Sistem eficient scalabil: functie de izoeficienta mica: ex - $O(p)$: dimensiunea problemei trebuie sa creasca liniar cu numarul de procesoare
- Sistem greu scalabil: functie de izoeficienta mare: ex - $O(2^p)$: dimensiunea problemei trebuie sa creasca exponential cu numarul de procesoare

Cum se definește dimensiunea problemei ?

- Definiția trebuie aleasă astfel încât dimensiunea să fie direct proporțională cu volumul calculelor
- **Dimensiunea problemei = W = numărul total de operații necesare rezolvării problemei**
- Exemplu: pentru înmulțirea a 2 matrici cu $n \times n$ elemente, dimensiunea problemei este $W = O(n^3)$
- **Dimensiunea problemei = $W = T_s$ = timpul de execuție al celui mai performant algoritmul secvențial**

Calculul functiei de izoeficienta

$$T_O = p \cdot T_P - T_S = p \cdot T_P - W$$

$$T_P = \frac{T_O + W}{p}$$

$$S = \frac{T_S}{T_P} = \frac{W \cdot p}{T_O + W}$$

$$E = \frac{S}{p} = \frac{W}{T_O + W} = E_0$$

$$W = \frac{E_0}{1 - E_0} \cdot T_O$$

$$K = \frac{E_0}{1 - E_0}$$

$$T_O = T_O(W, p)$$

$$W = K \cdot T_O(W, p)$$

prin manipulari algebricese extrage functia

$$W = f(p)$$

Exemplu - calcul functie izoeficienta

- Problema: adunarea a n numere pe p procesoare

$$T_p = \frac{n}{p} + 2 \cdot \log p$$

$$T_s = n$$

$$T_o = p \cdot T_p - T_s = 2 \cdot p \cdot \log p$$

$$W = \frac{E_0}{1 - E_0} \cdot T_o = K \cdot 2 \cdot p \cdot \log p$$

$W(p)$ exista pentru orice valoare $E_0 \Rightarrow$ alg scalabil
pentru a pastra eficienta constanta, daca numarul de
procesoare creste de la p la p' atunci dimensiunea datelor
(in cazul exemplului curent, n) trebuie sa creasca la

$$n' = n \cdot \frac{p' \cdot \log p'}{p \cdot \log p}$$

Exemplu - calcul functie izoeficienta

- **Problema: procesarea imaginilor $n \times n$ pe p procesoare**

$$T_S = 9 \cdot c \cdot n^2$$

$$T_P = 9 \cdot c \cdot \frac{n^2}{p} + 2 \cdot (t_s + t_w \cdot n)$$

$$T_O = p \cdot T_P - T_S = 2 \cdot p \cdot (t_s + t_w \cdot n)$$

$$W = K \cdot T_O = K \cdot 2 \cdot p \cdot (t_s + t_w \cdot n)$$

$$W = n^2$$

$$W = K \cdot 2 \cdot p \cdot (t_s + t_w \cdot \sqrt{W})$$

Pentru extragerea functiei $W = f(p)$:

se noteaza $x = \sqrt{W}$, rezulta o ecuatie de gradul 2 in x

$$x = \sqrt{W} = K \cdot p \cdot t_w + \sqrt{(K \cdot p \cdot t_w)^2 + K \cdot p \cdot t_s}$$

$$W(p) = O(p^2)$$

Exemplu - calcul functie izoeficienta (cont)

- Problema: procesarea imaginilor $n \times n$ pe p procesoare (con

$$W = n^2$$

$$W(p) = O(p^2)$$

$$p_1 = 2$$

$$n_1 = 8, W_1 = 64$$

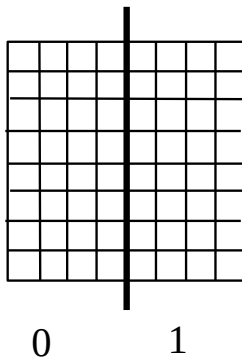
$$p_2 = 4$$

$$W_2 = W_1 \cdot \frac{p_2^2}{p_1^2} = 64 \cdot \frac{4 \cdot 4}{2 \cdot 2} = 256, n_2 = 16$$

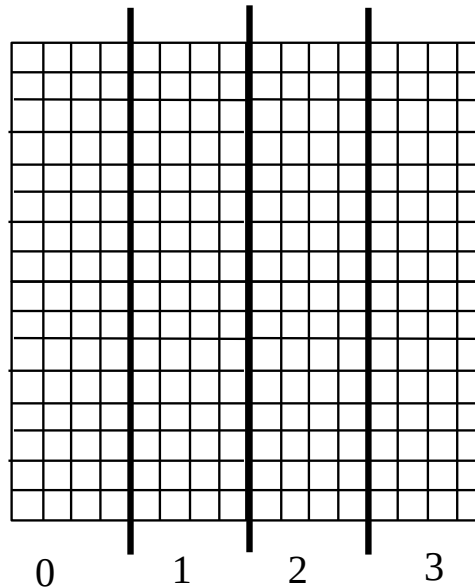
Exemplu - calcul functie izoeficienta

- Problema: procesarea imaginilor $n \times n$ pe p procesoare

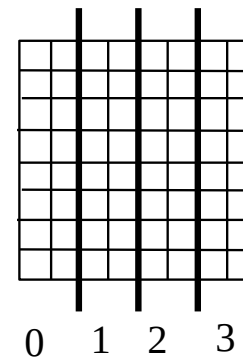
$p_1=2$
 $n_1=8, W_1=64$



$p_2=4$
 $n_2=16, W_2=256$
 $E_2=E_1$



$p_3=4$
 $n_3=8, W_3=64$
 $E_3 < E_1$



Calcul intuitiv functie izoefficienta

$$E = \frac{T_S}{p \cdot T_P} = \frac{\text{timpul de executie pe 1 procesor}}{\text{timpul de executie insumat al celor } p \text{ procesoare}}$$

$$E = \frac{T_S}{T_{\text{calcul}} + T_{\text{comunicatii}} + T_{\text{idle}}}$$

Conditie scalabilitate : $E = ct$

$\Rightarrow$ timpul de executie uniprosesor trebuie sa aiba aceeaasi rata de crestere
ca si timpul de executie paralel insumat

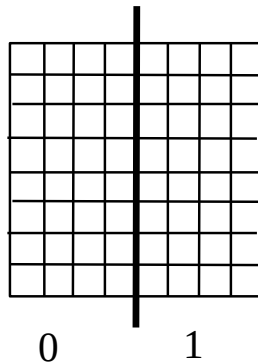
$$T_S \approx T_{\text{calcul}}$$

$\Rightarrow$ timpul de calcul efectiv trebuie sa aiba aceeaasi rata de crestere ca
si suprasarcina cauzata de comunicatii si inactivitati temporare

Exemplu - calcul intuitiv functie izoeficienta

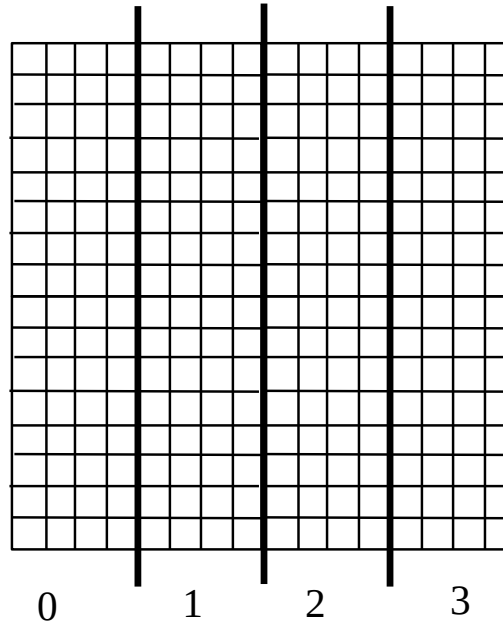
- Problema: procesarea imaginilor $n \times n$ pe p procesoare

$p_1=2$
 $n_1=8$



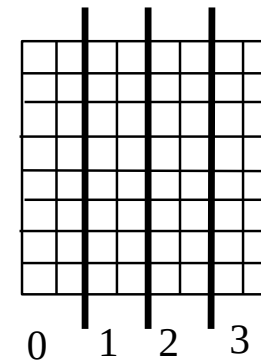
$T_{\text{calcul1}} = T_s = 8 \times 8$
 $T_{\text{comunic1}} = 2 \times 2 \times 8$

$p_2=4$
 $n_2=16$



$T_{\text{calcul2}} = 4 \times T_{\text{calcul1}}$
 $T_{\text{comunic2}} = 4 \times T_{\text{comunic1}}$
 $E_2 = E_1$

$p_3=4$
 $n_3=8$



$T_{\text{calcul3}} = 1 \times T_{\text{calcul1}}$
 $T_{\text{comunic3}} = 2 \times T_{\text{comunic1}}$
 $E_3 < E_1$

Relatia Cost-Optimalitate si Izoeficienta

- Un sistem este cost-optimal daca $p^*T_p = O(W)$
- $p^*T_p = T_o(p, W) + W$
- $\Rightarrow$ sistemul este cost-optimal daca $T_o(W, p) = O(W)$
- Echivalent cu ecuatia din care se calculeaza fct de izoeficienta: $W = K^*T_o(W, p)$

Timpul de executie minim pentru o anumita dimensiune a problemei

- Cat de repede poate fi rezolvata o problema de o dimensiune data, in conditiile in care nu se pun restrictii privitoare la numarul procesoarelor utilizate ?
- $p=?$ pentru a obtine T_{pmin}
- Fie p_0 o solutie a ecuatiei $dT_p/dp=0$
- $T_{pmin}=T_p(p_0)$

Exemplu de determinare a timpului de executie minim

- Adunarea a n numere pe p procesoare

$$T_P(n, p) = \frac{n}{p} + 2 \log p$$

$$\frac{dT_P}{dp} = -\frac{n}{p^2} + \frac{2}{p} = 0$$

$$-n + 2p = 0$$

$$p_0 = \frac{n}{2}$$

$$T_P^{\min} = T_P(p_0) = 2 \log n$$

$$Cost = T_P^{\min} \cdot p_0 = n \cdot \log n$$

Solutia nu este cost- optimala !

Timpul de executie cost-optimal

Adunarea a n numere pe p procesoare.

$$W = n = p \log p$$

$$p \approx \frac{n}{\log n}$$

$$T_p(n, p) = \frac{n}{p} + 2 \log p$$

$$T_p^{\text{cost_opt}} = \log n + 2 \log\left(\frac{n}{\log n}\right) = 2 \cdot \log n - \log \log n$$

$$T_p^{\text{cost_opt}} = O(\log n)$$

- Observatie: in acest exemplu in particular $T_P^{\min}$ si $T_P^{\text{cost_opt}}$ au rezultat $O(\log n)$.
- In general exista relatia $T_P^{\min} \leq T_P^{\text{cost_opt}}$